Introduction to MLIR

Session 1, Part 1 — CPU

SC26 Tutorial

Matrix-Matrix Multiplication in MLIR, Two Ways

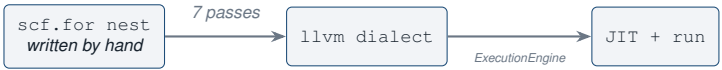
Today's example is matmul: `out[i, j] += sum_k a[i, k] * b[k, j]`

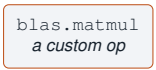


- First, write the matmul loop nest by hand.

Matrix-Matrix Multiplication in MLIR, Two Ways

Today's example is matmul: `out[i, j] += sum_k a[i, k] * b[k, j]`

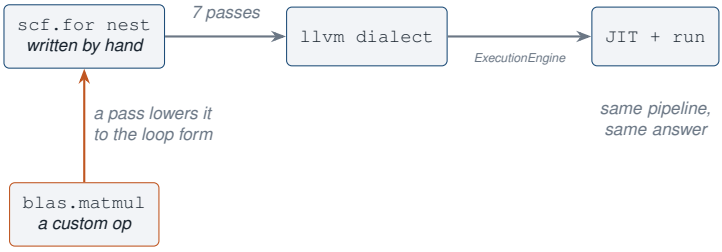




- First, write the matmul loop nest by hand.
- Then, define an operation that represents the matmul computation as a whole.

Matrix-Matrix Multiplication in MLIR, Two Ways

Today's example is matmul: `out[i, j] += sum_k a[i, k] * b[k, j]`



- First, write the matmul loop nest by hand.
- Then, define an operation that represents the matmul computation as a whole.
- Then, lower from that operation to the loop form.

Hands-On Exercises

- **The exercises are edits to MLIR and C++ files, in Colab.** Edit the file, save, and re-run the cells that build and run it.

Hands-On Exercises

- **The exercises are edits to MLIR and C++ files, in Colab.** Edit the file, save, and re-run the cells that build and run it.
- **We walk through each notebook cell together**, as a class.

Hands-On Exercises

- **The exercises are edits to MLIR and C++ files, in Colab.** Edit the file, save, and re-run the cells that build and run it.
- **We walk through each notebook cell together**, as a class.
- **For each exercise there is a solved matrix-*vector* example** to use as a reference.

Hands-On Exercises

- **The exercises are edits to MLIR and C++ files, in Colab.** Edit the file, save, and re-run the cells that build and run it.
- **We walk through each notebook cell together**, as a class.
- **For each exercise there is a solved matrix-*vector* example** to use as a reference.
- **Colab trick:** `Ctrl+F8` runs every cell above the current one, to catch up after losing track.

Step 0 — Setup

▶ Colab 1.2–1.8

Step 1 — Mat-Mult with Explicit Loops

Mat-Mult with Explicit Loops

A matrix-matrix product that accumulates into `out`:

```
out[i, j] += sum_k a[i, k] * b[k, j]
```

It goes in `session1/matmul_scf.mlir`, which ships empty:

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  // write the nested scf.for loops here
  return
}
```

Mat-Mult with Explicit Loops

A matrix-matrix product that accumulates into `out`:

```
out[i, j] += sum_k a[i, k] * b[k, j]
```

It goes in `session1/matmul_scf.mlir`, which ships empty:

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  // write the nested scf.for loops here
  return
}
```

- Use `matvec_scf.mlir` as the reference — same directory, same computation one dimension smaller, complete.

Mat-Mult with Explicit Loops

A matrix-matrix product that accumulates into `out`:

```
out[i, j] += sum_k a[i, k] * b[k, j]
```

It goes in `session1/matmul_scf.mlir`, which ships empty:

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  // write the nested scf.for loops here
  return
}
```

- Use `matvec_scf.mlir` as the reference — same directory, same computation one dimension smaller, complete.
- The goal: execute `@matmul` and get the expected answer.

Reference: `matvec_scf.mlir`, Step 1 of 3 — Bounds and Row Loop

```
func.func @matvec(%mat: memref<4x3xf32>, %vec: memref<3xf32>,
                  %out: memref<4xf32>) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  %numRows = memref.dim %mat, %c0 : memref<4x3xf32>
  %numCols = memref.dim %mat, %c1 : memref<4x3xf32>
  scf.for %i = %c0 to %numRows step %c1 {        // the row loop

    // the j-loop goes here: it reduces row i against %vec (next slide)

  }
  return
}
```

Reference: `matvec_scf.mlir`, Step 1 of 3 — Bounds and Row Loop

```
func.func @matvec(%mat: memref<4x3xf32>, %vec: memref<3xf32>,
                  %out: memref<4xf32>) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  %numRows = memref.dim %mat, %c0 : memref<4x3xf32>
  %numCols = memref.dim %mat, %c1 : memref<4x3xf32>
  scf.for %i = %c0 to %numRows step %c1 {        // the row loop

    // the j-loop goes here: it reduces row i against %vec (next slide)

  }
  return
}
```

- `memref.dim` retrieves the size of the buffer's dimensions.

Reference: `matvec_scf.mlir`, Step 1 of 3 — Bounds and Row Loop

```
func.func @matvec(%mat: memref<4x3xf32>, %vec: memref<3xf32>,
                  %out: memref<4xf32>) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  %numRows = memref.dim %mat, %c0 : memref<4x3xf32>
  %numCols = memref.dim %mat, %c1 : memref<4x3xf32>
  scf.for %i = %c0 to %numRows step %c1 {        // the row loop

    // the j-loop goes here: it reduces row i against %vec (next slide)

  }
  return
}
```

- `memref.dim` retrieves the size of the buffer's dimensions.
- The sizes are fixed, 4×3, only to keep the example simple. The bounds come from `memref.dim`, so the same loops work on `memref<?x?xf32>`, whose sizes are known only at run time.

Reference: `matvec_scf.mlir`, Step 2 of 3 — Reduction

```
// the j-loop: out[i] += sum_j mat[i, j] * vec[j]
%init = memref.load %out[%i] : memref<4xf32>
%acc = scf.for %j = %c0 to %numCols step %c1
                iter_args(%a = %init) -> (f32) {
  %m = memref.load %mat[%i, %j] : memref<4x3xf32>
  %v = memref.load %vec[%j] : memref<3xf32>
  %prod = arith.mulf %m, %v : f32
  %a2 = arith.addf %a, %prod : f32
  scf.yield %a2 : f32
}
```

- The `j`-loop is a reduction loop, with `%a` acting as the accumulator.

Reference: `matvec_scf.mlir`, Step 2 of 3 — Reduction

```
// the j-loop: out[i] += sum_j mat[i, j] * vec[j]
%init = memref.load %out[%i] : memref<4xf32>
%acc = scf.for %j = %c0 to %numCols step %c1
                iter_args(%a = %init) -> (f32) {
  %m = memref.load %mat[%i, %j] : memref<4x3xf32>
  %v = memref.load %vec[%j] : memref<3xf32>
  %prod = arith.mulf %m, %v : f32
  %a2 = arith.addf %a, %prod : f32
  scf.yield %a2 : f32
}
```

- The `j`-loop is a reduction loop, with `%a` acting as the accumulator.
- On the first iteration, `%a` is bound to `%init` — the current `out[i]`, so the sum starts from what `out` already holds.

Reference: `matvec_scf.mlir`, Step 2 of 3 — Reduction

```
// the j-loop: out[i] += sum_j mat[i, j] * vec[j]
%init = memref.load %out[%i] : memref<4xf32>
%acc = scf.for %j = %c0 to %numCols step %c1
                iter_args(%a = %init) -> (f32) {
  %m = memref.load %mat[%i, %j] : memref<4x3xf32>
  %v = memref.load %vec[%j] : memref<3xf32>
  %prod = arith.mulf %m, %v : f32
  %a2 = arith.addf %a, %prod : f32
  scf.yield %a2 : f32
}
```

- The `j`-loop is a reduction loop, with `%a` acting as the accumulator.
- On the first iteration, `%a` is bound to `%init` — the current `out[i]`, so the sum starts from what `out` already holds.
- Each iteration loads an element of `%mat` and of `%vec` and multiplies them: its contribution, `%prod`.

Reference: `matvec_scf.mlir`, Step 2 of 3 — Reduction

```
// the j-loop: out[i] += sum_j mat[i, j] * vec[j]
%init = memref.load %out[%i] : memref<4xf32>
%acc = scf.for %j = %c0 to %numCols step %c1
                iter_args(%a = %init) -> (f32) {
  %m = memref.load %mat[%i, %j] : memref<4x3xf32>
  %v = memref.load %vec[%j] : memref<3xf32>
  %prod = arith.mulf %m, %v : f32
  %a2 = arith.addf %a, %prod : f32
  scf.yield %a2 : f32
}
```

- The `j`-loop is a reduction loop, with `%a` acting as the accumulator.
- On the first iteration, `%a` is bound to `%init` — the current `out[i]`, so the sum starts from what `out` already holds.
- Each iteration loads an element of `%mat` and of `%vec` and multiplies them: its contribution, `%prod`.
- The body yields a value, and that value is bound to `%a` in the next iteration.

Reference: `matvec_scf.mlir`, Step 2 of 3 — Reduction

```
// the j-loop: out[i] += sum_j mat[i, j] * vec[j]
%init = memref.load %out[%i] : memref<4xf32>
%acc = scf.for %j = %c0 to %numCols step %c1
                iter_args(%a = %init) -> (f32) {
  %m = memref.load %mat[%i, %j] : memref<4x3xf32>
  %v = memref.load %vec[%j] : memref<3xf32>
  %prod = arith.mulf %m, %v : f32
  %a2 = arith.addf %a, %prod : f32
  scf.yield %a2 : f32
}
```

- The `j`-loop is a reduction loop, with `%a` acting as the accumulator.
- On the first iteration, `%a` is bound to `%init` — the current `out[i]`, so the sum starts from what `out` already holds.
- Each iteration loads an element of `%mat` and of `%vec` and multiplies them: its contribution, `%prod`.
- The body yields a value, and that value is bound to `%a` in the next iteration.
- The yield from the last iteration is the result of the loop: `%acc`.

Reference: `matvec_scf.mlir`, Step 3 of 3 — Store

```
func.func @matvec(%mat: memref<4x3xf32>, %vec: memref<3xf32>,
                  %out: memref<4xf32>) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  %numRows = memref.dim %mat, %c0 : memref<4x3xf32>
  %numCols = memref.dim %mat, %c1 : memref<4x3xf32>
  scf.for %i = %c0 to %numRows step %c1 {                    // the row loop
    %init = memref.load %out[%i] : memref<4xf32>
    %acc = scf.for %j = %c0 to %numCols step %c1             // the j-loop
                   iter_args(%a = %init) -> (f32) {
      %m = memref.load %mat[%i, %j] : memref<4x3xf32>
      %v = memref.load %vec[%j] : memref<3xf32>
      %prod = arith.mulf %m, %v : f32
      %a2 = arith.addf %a, %prod : f32
      scf.yield %a2 : f32
    }
    memref.store %acc, %out[%i] : memref<4xf32>              ← the value is written out
  }
  return
}
```

- `memref.store` writes the loop's result, `%acc`, out to `out[i]`.

Exercise 1 — Writing the Loop Nest

Fill in `@matmul` in `session1/matmul_scf.mlir`.

What is different from matvec:

- `b` and `out` are matrices now, so there is one more dimension to retrieve: the columns of `b`.
- One more loop, over the columns of `b`, between the row loop and the reduction loop.
- `out` and `b` take two indices: `out[i, j]` and `b[k, j]`.

Check after every save — re-run cell 1.12:

```
!mlir-opt matmul_scf.mlir
```

- Round-trip: `mlir-opt` parses and re-prints.
- A clean round-trip means it parses and verifies.

12 minutes to write it — then we walk through the solution.

Solution — `matmul_scf.mlir`

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  %M = memref.dim %a, %c0 : memref<4x3xf32>
  %K = memref.dim %a, %c1 : memref<4x3xf32>
  %N = memref.dim %b, %c1 : memref<3x5xf32>
  scf.for %i = %c0 to %M step %c1 {
    scf.for %j = %c0 to %N step %c1 {
      %init = memref.load %out[%i, %j] : memref<4x5xf32>
      %acc = scf.for %k = %c0 to %K step %c1
                     iter_args(%s = %init) -> (f32) {
        %av = memref.load %a[%i, %k] : memref<4x3xf32>
        %bv = memref.load %b[%k, %j] : memref<3x5xf32>
        %prod = arith.mulf %av, %bv : f32
        %s2 = arith.addf %s, %prod : f32
        scf.yield %s2 : f32
      }
      memref.store %acc, %out[%i, %j] : memref<4x5xf32>
    }
  }
  return
}
```

Solution — `matmul_scf.mlir`

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  %M = memref.dim %a, %c0 : memref<4x3xf32>
  %K = memref.dim %a, %c1 : memref<4x3xf32>                    ← find M, N and K
  %N = memref.dim %b, %c1 : memref<3x5xf32>
  scf.for %i = %c0 to %M step %c1 {
    scf.for %j = %c0 to %N step %c1 {
      %init = memref.load %out[%i, %j] : memref<4x5xf32>
      %acc = scf.for %k = %c0 to %K step %c1
                     iter_args(%s = %init) -> (f32) {
        %av = memref.load %a[%i, %k] : memref<4x3xf32>
        %bv = memref.load %b[%k, %j] : memref<3x5xf32>
        %prod = arith.mulf %av, %bv : f32
        %s2 = arith.addf %s, %prod : f32
        scf.yield %s2 : f32
      }
      memref.store %acc, %out[%i, %j] : memref<4x5xf32>
    }
  }
  return
}
```

Solution — `matmul_scf.mlir`

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  %M = memref.dim %a, %c0 : memref<4x3xf32>
  %K = memref.dim %a, %c1 : memref<4x3xf32>
  %N = memref.dim %b, %c1 : memref<3x5xf32>
  scf.for %i = %c0 to %M step %c1 {                    ← loop over rows and columns
    scf.for %j = %c0 to %N step %c1 {
      %init = memref.load %out[%i, %j] : memref<4x5xf32>
      %acc = scf.for %k = %c0 to %K step %c1
                     iter_args(%s = %init) -> (f32) {
        %av = memref.load %a[%i, %k] : memref<4x3xf32>
        %bv = memref.load %b[%k, %j] : memref<3x5xf32>
        %prod = arith.mulf %av, %bv : f32
        %s2 = arith.addf %s, %prod : f32
        scf.yield %s2 : f32
      }
      memref.store %acc, %out[%i, %j] : memref<4x5xf32>
    }
  }
  return
}
```

Solution — `matmul_scf.mlir`

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  %M = memref.dim %a, %c0 : memref<4x3xf32>
  %K = memref.dim %a, %c1 : memref<4x3xf32>
  %N = memref.dim %b, %c1 : memref<3x5xf32>
  scf.for %i = %c0 to %M step %c1 {
    scf.for %j = %c0 to %N step %c1 {
      %init = memref.load %out[%i, %j] : memref<4x5xf32>
      %acc = scf.for %k = %c0 to %K step %c1
                     iter_args(%s = %init) -> (f32) {
        %av = memref.load %a[%i, %k] : memref<4x3xf32>      ← read a and b
        %bv = memref.load %b[%k, %j] : memref<3x5xf32>
        %prod = arith.mulf %av, %bv : f32
        %s2 = arith.addf %s, %prod : f32
        scf.yield %s2 : f32
      }
      memref.store %acc, %out[%i, %j] : memref<4x5xf32> ← write the accumulated value to out
    }
  }
  return
}
```

Solution — `matmul_scf.mlir`

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  %c0 = arith.constant 0 : index
  %c1 = arith.constant 1 : index
  %M = memref.dim %a, %c0 : memref<4x3xf32>
  %K = memref.dim %a, %c1 : memref<4x3xf32>
  %N = memref.dim %b, %c1 : memref<3x5xf32>
  scf.for %i = %c0 to %M step %c1 {
    scf.for %j = %c0 to %N step %c1 {
      %init = memref.load %out[%i, %j] : memref<4x5xf32>
      %acc = scf.for %k = %c0 to %K step %c1
                     iter_args(%s = %init) -> (f32) {
        %av = memref.load %a[%i, %k] : memref<4x3xf32>
        %bv = memref.load %b[%k, %j] : memref<3x5xf32>
        %prod = arith.mulf %av, %bv : f32
        %s2 = arith.addf %s, %prod : f32
        scf.yield %s2 : f32
      }
      memref.store %acc, %out[%i, %j] : memref<4x5xf32>
    }
  }
  return
}
```

▶ Colab 1.12, 1.14

The Lowering Pipeline

```
func.func(llvm-request-c-wrappers)
convert-scf-to-cf
convert-cf-to-llvm
convert-arith-to-llvm
finalize-memref-to-llvm
convert-func-to-llvm
reconcile-unrealized-casts
```

- Seven passes, each lowering one dialect.

The Lowering Pipeline

```
func.func(llvm-request-c-wrappers)    ← a C-callable @matmul wrapper, for Python
convert-scf-to-cf
convert-cf-to-llvm
convert-arith-to-llvm
finalize-memref-to-llvm
convert-func-to-llvm
reconcile-unrealized-casts
```

- Seven passes, each lowering one dialect.

The Lowering Pipeline

```
func.func(llvm-request-c-wrappers)
convert-scf-to-cf                    ← structured loops become a CFG
convert-cf-to-llvm
convert-arith-to-llvm
finalize-memref-to-llvm
convert-func-to-llvm
reconcile-unrealized-casts
```

- Seven passes, each lowering one dialect.

The Lowering Pipeline

```
func.func(llvm-request-c-wrappers)
convert-scf-to-cf
convert-cf-to-llvm                    ← branches become llvm.br / llvm.cond_br
convert-arith-to-llvm
finalize-memref-to-llvm
convert-func-to-llvm
reconcile-unrealized-casts
```

- Seven passes, each lowering one dialect.

The Lowering Pipeline

```
func.func(llvm-request-c-wrappers)
convert-scf-to-cf
convert-cf-to-llvm
convert-arith-to-llvm                ← mulf, addf
finalize-memref-to-llvm
convert-func-to-llvm
reconcile-unrealized-casts
```

- Seven passes, each lowering one dialect.

The Lowering Pipeline

```
func.func(llvm-request-c-wrappers)
convert-scf-to-cf
convert-cf-to-llvm
convert-arith-to-llvm
finalize-memref-to-llvm                ← loads become getelementptr + load
convert-func-to-llvm
reconcile-unrealized-casts
```

- Seven passes, each lowering one dialect.

The Lowering Pipeline

```
func.func(llvm-request-c-wrappers)
convert-scf-to-cf
convert-cf-to-llvm
convert-arith-to-llvm
finalize-memref-to-llvm
convert-func-to-llvm                ← signatures, calling convention
reconcile-unrealized-casts
```

- Seven passes, each lowering one dialect.

The Lowering Pipeline

```
func.func(llvm-request-c-wrappers)
convert-scf-to-cf
convert-cf-to-llvm
convert-arith-to-llvm
finalize-memref-to-llvm
convert-func-to-llvm
reconcile-unrealized-casts            ← clean up the type-conversion seams
```

- Seven passes, each lowering one dialect.

The Lowering Pipeline

```
func.func(llvm-request-c-wrappers)
convert-scf-to-cf
convert-cf-to-llvm
convert-arith-to-llvm
finalize-memref-to-llvm
convert-func-to-llvm
reconcile-unrealized-casts
```

- Seven passes, each lowering one dialect.
- After every one of them the IR is a valid MLIR module, which can be printed and read.

```
mlir-opt ...  --mlir-print-ir-after-all --mlir-disable-threading
```

The Lowering Pipeline

```
func.func(llvm-request-c-wrappers)
convert-scf-to-cf
convert-cf-to-llvm
convert-arith-to-llvm
finalize-memref-to-llvm
convert-func-to-llvm
reconcile-unrealized-casts
```

- Seven passes, each lowering one dialect.
- After every one of them the IR is a valid MLIR module, which can be printed and read.

```
mlir-opt ...  --mlir-print-ir-after-all --mlir-disable-threading
```

- The result is MLIR's `llvm` *dialect* — not LLVM IR. Translation to real LLVM IR happens inside the execution engine.

Execution

```
a = np.arange(12, dtype=np.float32).reshape(4, 3)
b = np.arange(15, dtype=np.float32).reshape(3, 5)
out = np.zeros((4, 5), dtype=np.float32)

run_and_check(module, "matmul", [a, b, out], expected=a @ b)
```

Execution

```
a = np.arange(12, dtype=np.float32).reshape(4, 3)
b = np.arange(15, dtype=np.float32).reshape(3, 5)
out = np.zeros((4, 5), dtype=np.float32)

run_and_check(module, "matmul", [a, b, out], expected=a @ b)
```

- numpy arrays are passed as memref descriptors — a struct of pointer, offset, sizes and strides. No copying.

Execution

```
a = np.arange(12, dtype=np.float32).reshape(4, 3)
b = np.arange(15, dtype=np.float32).reshape(3, 5)
out = np.zeros((4, 5), dtype=np.float32)

run_and_check(module, "matmul", [a, b, out], expected=a @ b)
```

- numpy arrays are passed as memref descriptors — a struct of pointer, offset, sizes and strides. No copying.
- The last argument is the output buffer, by convention. `run_and_check` zeroes it again before every run, because the notebook reuses it from step to step.

Execution

```
a = np.arange(12, dtype=np.float32).reshape(4, 3)
b = np.arange(15, dtype=np.float32).reshape(3, 5)
out = np.zeros((4, 5), dtype=np.float32)

run_and_check(module, "matmul", [a, b, out], expected=a @ b)
```

- numpy arrays are passed as memref descriptors — a struct of pointer, offset, sizes and strides. No copying.
- The last argument is the output buffer, by convention. `run_and_check` zeroes it again before every run, because the notebook reuses it from step to step.
- Checked against numpy's own matmul.

Execution

```
a = np.arange(12, dtype=np.float32).reshape(4, 3)
b = np.arange(15, dtype=np.float32).reshape(3, 5)
out = np.zeros((4, 5), dtype=np.float32)

run_and_check(module, "matmul", [a, b, out], expected=a @ b)
```

- numpy arrays are passed as memref descriptors — a struct of pointer, offset, sizes and strides. No copying.
- The last argument is the output buffer, by convention. `run_and_check` zeroes it again before every run, because the notebook reuses it from step to step.
- Checked against numpy's own matmul.

```
OK
```

Step 1 Recap

A matrix multiply written at the loop level,
compiled through seven passes, and run.

Step 1 Recap

A matrix multiply written at the loop level,
compiled through seven passes, and run.

- `scf.for` with `iter_args` for the reduction.

Step 1 Recap

A matrix multiply written at the loop level,
compiled through seven passes, and run.

- `scf.for` with `iter_args` for the reduction.
- `memref.dim` so the bounds come from the buffer.

Step 1 Recap

A matrix multiply written at the loop level,
compiled through seven passes, and run.

- `scf.for` with `iter_args` for the reduction.
- `memref.dim` so the bounds come from the buffer.
- One pipeline, dialect by dialect, down to `llvm`.

Step 1 Recap

A matrix multiply written at the loop level,
compiled through seven passes, and run.

- `scf.for` with `iter_args` for the reduction.
- `memref.dim` so the bounds come from the buffer.
- One pipeline, dialect by dialect, down to `llvm`.

Short break. If anything is not working, raise a hand.

Step 2 — `blas` Dialect

Defining a New Level of Abstraction

the loop nest

```
scf.for %i = ... {
  scf.for %j = ... {
    %0 = memref.load %out[%i, %j]
    %1 = scf.for %k = ... iter_args(...) {
      ...
    }
    memref.store %1, %out[%i, %j]
  }
}
```

20 operations.

Defining a New Level of Abstraction

the loop nest

```
scf.for %i = ... {
  scf.for %j = ... {
    %0 = memref.load %out[%i, %j]
    %1 = scf.for %k = ... iter_args(...) {
      ...
    }
    memref.store %1, %out[%i, %j]
  }
}
```

20 operations.

an operation that represents it

```
blas.matmul %a, %b, %out
  : memref<4x3xf32>,
    memref<3x5xf32>,
    memref<4x5xf32>
```

One operation.

Defining a New Level of Abstraction

the loop nest

```
scf.for %i = ... {
  scf.for %j = ... {
    %0 = memref.load %out[%i, %j]
    %1 = scf.for %k = ... iter_args(...) {
      ...
    }
    memref.store %1, %out[%i, %j]
  }
}
```

20 operations.

an operation that represents it

```
blas.matmul %a, %b, %out
  : memref<4x3xf32>,
    memref<3x5xf32>,
    memref<4x5xf32>
```

One operation.

- MLIR supports higher levels of abstraction — a custom DSL — as new operations.

Defining a New Level of Abstraction

the loop nest

```
scf.for %i = ... {
  scf.for %j = ... {
    %0 = memref.load %out[%i, %j]
    %1 = scf.for %k = ... iter_args(...) {
      ...
    }
    memref.store %1, %out[%i, %j]
  }
}
```

20 operations.

an operation that represents it

```
blas.matmul %a, %b, %out
  : memref<4x3xf32>,
    memref<3x5xf32>,
    memref<4x5xf32>
```

One operation.

- MLIR supports higher levels of abstraction — a custom DSL — as new operations.
- The compiler then reasons about a matmul operation directly, instead of deriving that from a loop nest by pattern matching. Nothing has to be lifted back out of the loops.

Defining a New Level of Abstraction

the loop nest

```
scf.for %i = ... {
  scf.for %j = ... {
    %0 = memref.load %out[%i, %j]
    %1 = scf.for %k = ... iter_args(...) {
      ...
    }
    memref.store %1, %out[%i, %j]
  }
}
```

20 operations.

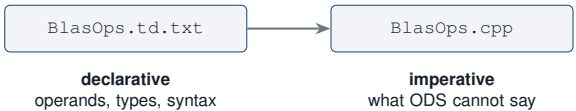
an operation that represents it

```
blas.matmul %a, %b, %out
  : memref<4x3xf32>,
    memref<3x5xf32>,
    memref<4x5xf32>
```

One operation.

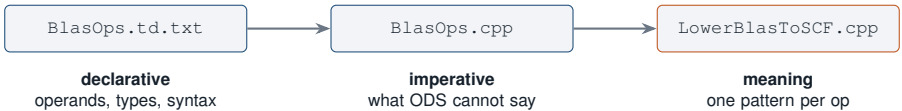
- MLIR supports higher levels of abstraction — a custom DSL — as new operations.
- The compiler then reasons about a matmul operation directly, instead of deriving that from a loop nest by pattern matching. Nothing has to be lifted back out of the loops.
- One definition, many lowerings: CPU loops in this part, `gpu.launch` in Part 2.

Anatomy of a Dialect



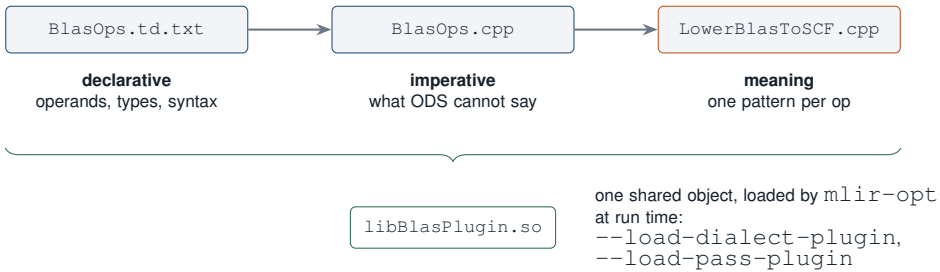
- First, define the op. Its ODS (Operation Definition Specification) is a declarative spec written in TableGen. `mlir-tblgen` generates the op's C++ from it, and the verifier in `BlasOps.cpp` adds what ODS cannot say.

Anatomy of a Dialect



- First, define the op. Its ODS (Operation Definition Specification) is a declarative spec written in TableGen. `mlir-tblgen` generates the op's C++ from it, and the verifier in `BlasOps.cpp` adds what ODS cannot say.
- Then, write the lowering that turns it into loops.

Anatomy of a Dialect



- First, define the op. Its ODS (Operation Definition Specification) is a declarative spec written in TableGen. `mlir-tblgen` generates the op's C++ from it, and the verifier in `BlasOps.cpp` adds what ODS cannot say.
- Then, write the lowering that turns it into loops.

Defining an Operation

We want to define an operation that looks like this:

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  blas.matmul %a, %b, %out
    : memref<4x3xf32>, memref<3x5xf32>, memref<4x5xf32>
  return
}
```

Defining an Operation

We want to define an operation that looks like this:

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  blas.matmul %a, %b, %out
    : memref<4x3xf32>, memref<3x5xf32>, memref<4x5xf32>
  return
}
```

- That is `session1/matmul_custom.mlir` — the MLIR format we expect is in that file, already written.

Defining an Operation

We want to define an operation that looks like this:

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  blas.matmul %a, %b, %out
    : memref<4x3xf32>, memref<3x5xf32>, memref<4x5xf32>
  return
}
```

- That is `session1/matmul_custom.mlir` — the MLIR format we expect is in that file, already written.
- Round-tripping it would fail: the operation is not defined yet.

```
matmul_custom.mlir:5:3: error: custom op 'blas.matmul' is unknown
```

Defining an Operation

We want to define an operation that looks like this:

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  blas.matmul %a, %b, %out
    : memref<4x3xf32>, memref<3x5xf32>, memref<4x5xf32>
  return
}
```

- That is `session1/matmul_custom.mlir` — the MLIR format we expect is in that file, already written.
- Round-tripping it would fail: the operation is not defined yet.

```
matmul_custom.mlir:5:3: error: custom op 'blas.matmul' is unknown
```

- Once the operation is defined, the same round-trip succeeds: `mlir-opt` parses the file and prints it back in the same syntax.

Reference: `Blas_MatVecOp`, Step 1 of 5 — Operands

blas-dialect/include/Blas/BlasOps.td.txt:

```
def Blas_MatVecOp : Blas_Op<"matvec"> {
  let summary = "matrix-vector product: out += mat * vec";

  let arguments = (ins
    MemRefRankOf<[F32], [2]>:$mat,
    MemRefRankOf<[F32], [1]>:$vec,
    MemRefRankOf<[F32], [1]>:$out);

  // steps 2 and 4: assemblyFormat and hasVerifier
}
```

Reference: `Blas_MatVecOp`, Step 1 of 5 — Operands

`blas-dialect/include/Blas/BlasOps.td.txt`:

```
def Blas_MatVecOp : Blas_Op<"matvec"> {
  let summary = "matrix-vector product: out += mat * vec";

  let arguments = (ins
    MemRefRankOf<[F32], [2]>:$mat,
    MemRefRankOf<[F32], [1]>:$vec,
    MemRefRankOf<[F32], [1]>:$out);

  // steps 2 and 4: assemblyFormat and hasVerifier
}
```

- `Blas_Op<"matvec">` makes this the op `blas.matvec`. Its C++ class is the def name minus the `Blas_` prefix: `blas::MatVecOp`.

Reference: `Blas_MatVecOp`, Step 1 of 5 — Operands

`blas-dialect/include/Blas/BlasOps.td.txt`:

```
def Blas_MatVecOp : Blas_Op<"matvec"> {
  let summary = "matrix-vector product: out += mat * vec";

  let arguments = (ins
    MemRefRankOf<[F32], [2]>:$mat,          ← a 2D f32 memref, named mat
    MemRefRankOf<[F32], [1]>:$vec,
    MemRefRankOf<[F32], [1]>:$out);

  // steps 2 and 4: assemblyFormat and hasVerifier
}
```

- `Blas_Op<"matvec">` makes this the op `blas.matvec`. Its C++ class is the def name minus the `Blas_` prefix: `blas::MatVecOp`.
- `arguments` lists the operands. Each one has a type constraint and a name.

Reference: `Blas_MatVecOp`, Step 1 of 5 — Operands

`blas-dialect/include/Blas/BlasOps.td.txt`:

```
def Blas_MatVecOp : Blas_Op<"matvec"> {
  let summary = "matrix-vector product: out += mat * vec";

  let arguments = (ins
    MemRefRankOf<[F32], [2]>:$mat,
    MemRefRankOf<[F32], [1]>:$vec,
    MemRefRankOf<[F32], [1]>:$out);

  // steps 2 and 4: assemblyFormat and hasVerifier
}
```

- `Blas_Op<"matvec">` makes this the op `blas.matvec`. Its C++ class is the def name minus the `Blas_` prefix: `blas::MatVecOp`.
- `arguments` lists the operands. Each one has a type constraint and a name.
- ODS generates the check for those types: a `blas.matvec` with a 1D `mat` is rejected with no hand-written C++.

Reference: `Blas_MatVecOp`, Step 2 of 5 — Assembly Format

```
let assemblyFormat =
  "$mat `,` $vec `,` $out attr-dict `:` type($mat) `,` type($vec) `,` type($out)";
```

- From it, TableGen generates both the parser (text to op) and the printer (op to text), so the two always agree.

Reference: `Blas_MatVecOp`, Step 2 of 5 — Assembly Format

```
let assemblyFormat =
  "$mat `,` $vec `,` $out attr-dict `:` type($mat) `,` type($vec) `,` type($out)";
```

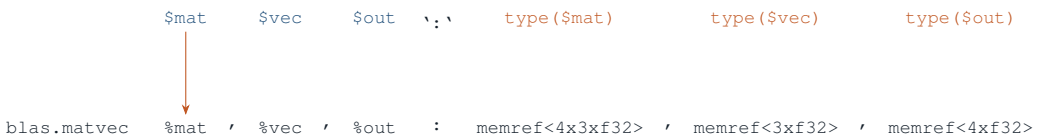
- From it, TableGen generates both the parser (text to op) and the printer (op to text), so the two always agree.

	$mat	$vec	$out	`:`	type($mat)	type($vec)	type($out)
blas.matvec	%mat	, %vec	, %out	:	memref<4x3xf32>	, memref<3xf32>	, memref<4xf32>

Reference: `Blas_MatVecOp`, Step 2 of 5 — Assembly Format

```
let assemblyFormat =
  "$mat `,` $vec `,` $out attr-dict `:` type($mat) `,` type($vec) `,` type($out)";
```

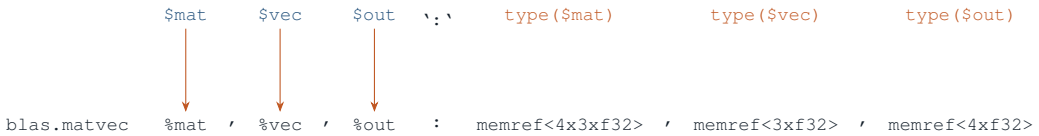
- From it, TableGen generates both the parser (text to op) and the printer (op to text), so the two always agree.



Reference: `Blas_MatVecOp`, Step 2 of 5 — Assembly Format

```
let assemblyFormat =
  "$mat `,` $vec `,` $out attr-dict `:` type($mat) `,` type($vec) `,` type($out)";
```

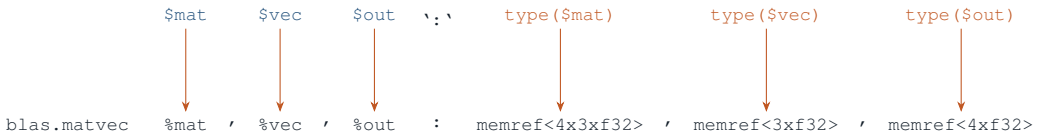
- From it, TableGen generates both the parser (text to op) and the printer (op to text), so the two always agree.



Reference: `Blas_MatVecOp`, Step 2 of 5 — Assembly Format

```
let assemblyFormat =
  "$mat `,` $vec `,` $out attr-dict `:` type($mat) `,` type($vec) `,` type($out)";
```

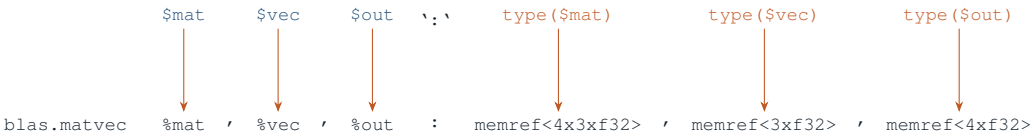
- From it, TableGen generates both the parser (text to op) and the printer (op to text), so the two always agree.



Reference: `Blas_MatVecOp`, Step 2 of 5 — Assembly Format

```
let assemblyFormat =
  "$mat `,` $vec `,` $out attr-dict `:` type($mat) `,` type($vec) `,` type($out)";
```

- From it, TableGen generates both the parser (text to op) and the printer (op to text), so the two always agree.



- `attr-dict` before the colon is not optional boilerplate. Leave it out and the op cannot carry attributes, which breaks things later in ways that look unrelated.

Reference: `Blas_MatVecOp`, Step 3 of 5 — TableGen

From the ODS, `mlir-tblgen` writes `build/include/Blas/BlasOps.h.inc`:

```
class MatVecOp : public ::mlir::Op<MatVecOp, ...> {
  ::mlir::TypedValue<::mlir::MemRefType> getMat();
  ::mlir::TypedValue<::mlir::MemRefType> getVec();
  ::mlir::TypedValue<::mlir::MemRefType> getOut();
  static void build(::mlir::OpBuilder &odsBuilder, ::mlir::OperationState &odsState,
                    ::mlir::Value mat, ::mlir::Value vec, ::mlir::Value out);

  ::llvm::LogicalResult verify();

};
```

Reference: `Blas_MatVecOp`, Step 3 of 5 — TableGen

From the ODS, `mlir-tblgen` writes `build/include/Blas/BlasOps.h.inc`:

```
class MatVecOp : public ::mlir::Op<MatVecOp, ...> {
  ::mlir::TypedValue<::mlir::MemRefType> getMat();        ← from $mat
  ::mlir::TypedValue<::mlir::MemRefType> getVec();        ← from $vec
  ::mlir::TypedValue<::mlir::MemRefType> getOut();        ← from $out
  static void build(::mlir::OpBuilder &odsBuilder, ::mlir::OperationState &odsState,
                    ::mlir::Value mat, ::mlir::Value vec, ::mlir::Value out);

  ::llvm::LogicalResult verify();

};
```

- Every operand named in `arguments` gets a `get*` accessor:
  - `$mat` used to generate `getMat()`
  - `$out` used to generate `getOut()`

Reference: `Blas_MatVecOp`, Step 3 of 5 — TableGen

From the ODS, `mlir-tblgen` writes `build/include/Blas/BlasOps.h.inc`:

```
class MatVecOp : public ::mlir::Op<MatVecOp, ...> {
  ::mlir::TypedValue<::mlir::MemRefType> getMat();
  ::mlir::TypedValue<::mlir::MemRefType> getVec();
  ::mlir::TypedValue<::mlir::MemRefType> getOut();
  static void build(::mlir::OpBuilder &odsBuilder, ::mlir::OperationState &odsState,
                    ::mlir::Value mat, ::mlir::Value vec, ::mlir::Value out);
                              ← what a pattern calls to CREATE the op
  ::llvm::LogicalResult verify();

};
```

- Every operand named in `arguments` gets a `get*` accessor:
  - `$mat` used to generate `getMat()`
  - `$out` used to generate `getOut()`

Reference: `Blas_MatVecOp`, Step 3 of 5 — TableGen

From the ODS, `mlir-tblgen` writes `build/include/Blas/BlasOps.h.inc`:

```
class MatVecOp : public ::mlir::Op<MatVecOp, ...> {
  ::mlir::TypedValue<::mlir::MemRefType> getMat();
  ::mlir::TypedValue<::mlir::MemRefType> getVec();
  ::mlir::TypedValue<::mlir::MemRefType> getOut();
  static void build(::mlir::OpBuilder &odsBuilder, ::mlir::OperationState &odsState,
                    ::mlir::Value mat, ::mlir::Value vec, ::mlir::Value out);

  ::llvm::LogicalResult verify();
                              ← declared here, defined in step 5
};
```

- Every operand named in `arguments` gets a `get*` accessor:
  - `$mat` used to generate `getMat()`
  - `$out` used to generate `getOut()`

Reference: `Blas_MatVecOp`, Step 3 of 5 — TableGen

From the ODS, `mlir-tblgen` writes `build/include/Blas/BlasOps.h.inc`:

```
class MatVecOp : public ::mlir::Op<MatVecOp, ...> {
  ::mlir::TypedValue<::mlir::MemRefType> getMat();
  ::mlir::TypedValue<::mlir::MemRefType> getVec();
  ::mlir::TypedValue<::mlir::MemRefType> getOut();
  static void build(::mlir::OpBuilder &odsBuilder, ::mlir::OperationState &odsState,
                    ::mlir::Value mat, ::mlir::Value vec, ::mlir::Value out);

  ::llvm::LogicalResult verify();

};
```

- Every operand named in `arguments` gets a `get*` accessor:
  - `$mat` used to generate `getMat()`
  - `$out` used to generate `getOut()`
- ODS generates the boilerplate — accessors, builders, parser, printer — and keeps operations, dialects and types consistent across MLIR.

Reference: `Blas_MatVecOp`, Step 3 of 5 — TableGen

From the ODS, `mlir-tblgen` writes `build/include/Blas/BlasOps.h.inc`:

```
class MatVecOp : public ::mlir::Op<MatVecOp, ...> {
  ::mlir::TypedValue<::mlir::MemRefType> getMat();
  ::mlir::TypedValue<::mlir::MemRefType> getVec();
  ::mlir::TypedValue<::mlir::MemRefType> getOut();
  static void build(::mlir::OpBuilder &odsBuilder, ::mlir::OperationState &odsState,
                    ::mlir::Value mat, ::mlir::Value vec, ::mlir::Value out);

  ::llvm::LogicalResult verify();

};
```

- Every operand named in `arguments` gets a `get*` accessor:
  - `$mat` used to generate `getMat()`
  - `$out` used to generate `getOut()`
- ODS generates the boilerplate — accessors, builders, parser, printer — and keeps operations, dialects and types consistent across MLIR.
- Nothing in this file is edited by hand. It is regenerated on every build.

Reference: `Blas_MatVecOp`, Step 4 of 5 — ODS

- ODS can say “a 2D f32 memref and two 1D ones”.

Reference: `Blas_MatVecOp`, Step 4 of 5 — ODS

- ODS can say “a 2D f32 memref and two 1D ones”.
- ODS cannot say “`mat`’s column count equals `vec`’s size”. Shape *relationships* go in the C++ `verify()` method:

constraint	why
`mat.dim(1) == vec.dim(0)`	the reduction dimension
`mat.dim(0) == out.dim(0)`	rows

Reference: `Blas_MatVecOp`, Step 4 of 5 — ODS

- ODS can say "a 2D f32 memref and two 1D ones".
- ODS cannot say "`mat`'s column count equals `vec`'s size". Shape *relationships* go in the C++ `verify()` method:

constraint	why
`mat.dim(1) == vec.dim(0)`	the reduction dimension
`mat.dim(0) == out.dim(0)`	rows

- `let hasVerifier = 1;` in the ODS is what generates the `verify()` *declaration*. Leave out the C++ definition and `mlir-opt` fails to load the plugin with an undefined-symbol error.

Reference: `Blas_MatVecOp`, Step 4 of 5 — ODS

- ODS can say "a 2D f32 memref and two 1D ones".
- ODS cannot say "`mat`'s column count equals `vec`'s size". Shape *relationships* go in the C++ `verify()` method:

constraint	why
`mat.dim(1) == vec.dim(0)`	the reduction dimension
`mat.dim(0) == out.dim(0)`	rows

- `let hasVerifier = 1;` in the ODS is what generates the `verify()` *declaration*. Leave out the C++ definition and `mlir-opt` fails to load the plugin with an undefined-symbol error.
- The ODS checks plus the hand-written `verify()` are the op's verifier. MLIR runs it when IR is parsed and after every pass, so the op's guarantees hold on the input and across every transformation.

Reference: `Blas_MatVecOp`, Step 5 of 5 — `MatVecOp::verify()`

blas-dialect/lib/BlasOps.cpp:

```
LogicalResult MatVecOp::verify() {
  auto matType = cast<MemRefType>(getMat().getType());
  auto vecType = cast<MemRefType>(getVec().getType());
  auto outType = cast<MemRefType>(getOut().getType());

  if (failed(verifyDimsMatch(*this, matType.getDimSize(1),
                             vecType.getDimSize(0),
                             "matrix column count must match input vector size")))
    return failure();

  return verifyDimsMatch(*this, matType.getDimSize(0), outType.getDimSize(0),
                         "matrix row count must match output vector size");
}
```

Reference: `Blas_MatVecOp`, Step 5 of 5 — `MatVecOp::verify()`

blas-dialect/lib/BlasOps.cpp:

```
LogicalResult MatVecOp::verify() {
  auto matType = cast<MemRefType>(getMat().getType());          ← the generated accessors
  auto vecType = cast<MemRefType>(getVec().getType());
  auto outType = cast<MemRefType>(getOut().getType());

  if (failed(verifyDimsMatch(*this, matType.getDimSize(1),
                             vecType.getDimSize(0),
                             "matrix column count must match input vector size")))
    return failure();

  return verifyDimsMatch(*this, matType.getDimSize(0), outType.getDimSize(0),
                         "matrix row count must match output vector size");
}
```

- `getMat()`, `getVec()` and `getOut()` are the accessors from step 3.

Reference: `Blas_MatVecOp`, Step 5 of 5 — `MatVecOp::verify()`

`blas-dialect/lib/BlasOps.cpp`:

```
LogicalResult MatVecOp::verify() {
  auto matType = cast<MemRefType>(getMat().getType());
  auto vecType = cast<MemRefType>(getVec().getType());
  auto outType = cast<MemRefType>(getOut().getType());

  if (failed(verifyDimsMatch(*this, matType.getDimSize(1),      ← the reduction dimension
                             vecType.getDimSize(0),
                             "matrix column count must match input vector size")))
    return failure();

  return verifyDimsMatch(*this, matType.getDimSize(0), outType.getDimSize(0),
                         "matrix row count must match output vector size");
}
```

- `getMat()`, `getVec()` and `getOut()` are the accessors from step 3.
- One `verifyDimsMatch` call per constraint. The helper lets a dynamic (`?`) dimension through: its size is known only at run time, so the verifier cannot check it.

Reference: `Blas_MatVecOp`, Step 5 of 5 — `MatVecOp::verify()`

blas-dialect/lib/BlasOps.cpp:

```
LogicalResult MatVecOp::verify() {
  auto matType = cast<MemRefType>(getMat().getType());
  auto vecType = cast<MemRefType>(getVec().getType());
  auto outType = cast<MemRefType>(getOut().getType());

  if (failed(verifyDimsMatch(*this, matType.getDimSize(1),
                             vecType.getDimSize(0),
                             "matrix column count must match input vector size")))
    return failure();

  return verifyDimsMatch(*this, matType.getDimSize(0), outType.getDimSize(0),   ← rows
                         "matrix row count must match output vector size");
}
```

- `getMat()`, `getVec()` and `getOut()` are the accessors from step 3.
- One `verifyDimsMatch` call per constraint. The helper lets a dynamic (`?`) dimension through: its size is known only at run time, so the verifier cannot check it.

Reference: `Blas_MatVecOp`, Step 5 of 5 — `MatVecOp::verify()`

blas-dialect/lib/BlasOps.cpp:

```
LogicalResult MatVecOp::verify() {
  auto matType = cast<MemRefType>(getMat().getType());
  auto vecType = cast<MemRefType>(getVec().getType());
  auto outType = cast<MemRefType>(getOut().getType());

  if (failed(verifyDimsMatch(*this, matType.getDimSize(1),
                             vecType.getDimSize(0),
                             "matrix column count must match input vector size")))
    return failure();

  return verifyDimsMatch(*this, matType.getDimSize(0), outType.getDimSize(0),
                         "matrix row count must match output vector size");
}
```

- `getMat()`, `getVec()` and `getOut()` are the accessors from step 3.
- One `verifyDimsMatch` call per constraint. The helper lets a dynamic (`?`) dimension through: its size is known only at run time, so the verifier cannot check it.
- The string is the op's own error message — what someone sees when they write a `blas.matvec` whose shapes do not agree.

Exercise 2 — Defining the Op

2a. The ODS `blas-dialect/include/Blas/BlasOps.td.txt`

- Define `Blas_MatMulOp` where the `TODO` is, modelled on `Blas_MatVecOp` right above it.
- Three 2D `f32` memrefs, named `$a`, `$b` and `$out`. Include `let hasVerifier = 1;`

2b. The verifier `blas-dialect/lib/BlasOps.cpp`

- Write `LogicalResult MatMulOp::verify()`, modelled on `MatVecOp::verify()`.
- Three constraints, one `verifyDimsMatch` call each.

These two go together: `hasVerifier = 1` declares `verify()`, and 2b defines it.

12 minutes to write them — then we walk through the solutions.

Solution — ODS

```
def Blas_MatMulOp : Blas_Op<"matmul"> {
  let summary = "matrix-matrix product: out += a * b";

  let arguments = (ins
    MemRefRankOf<[F32], [2]>:$a,
    MemRefRankOf<[F32], [2]>:$b,
    MemRefRankOf<[F32], [2]>:$out);

  let assemblyFormat =
    "$a `,` $b `,` $out attr-dict `:` type($a) `,` type($b) `,` type($out)";

  let hasVerifier = 1;
}
```

Solution — ODS

```
def Blas_MatMulOp : Blas_Op<"matmul"> {                    ← a new name
  let summary = "matrix-matrix product: out += a * b";

  let arguments = (ins
    MemRefRankOf<[F32], [2]>:$a,
    MemRefRankOf<[F32], [2]>:$b,
    MemRefRankOf<[F32], [2]>:$out);

  let assemblyFormat =
    "$a `,` $b `,` $out attr-dict `:` type($a) `,` type($b) `,` type($out)";

  let hasVerifier = 1;
}
```

Solution — ODS

```
def Blas_MatMulOp : Blas_Op<"matmul"> {
  let summary = "matrix-matrix product: out += a * b";

  let arguments = (ins
    MemRefRankOf<[F32], [2]>:$a,                    ← new operand names
    MemRefRankOf<[F32], [2]>:$b,
    MemRefRankOf<[F32], [2]>:$out);

  let assemblyFormat =                              ← the same format, with the new names
    "$a `,` $b `,` $out attr-dict `:` type($a) `,` type($b) `,` type($out)";

  let hasVerifier = 1;
}
```

Solution — ODS

```
def Blas_MatMulOp : Blas_Op<"matmul"> {
  let summary = "matrix-matrix product: out += a * b";

  let arguments = (ins
    MemRefRankOf<[F32], [2]>:$a,
    MemRefRankOf<[F32], [2]>:$b,                    ← b is a matrix now ...
    MemRefRankOf<[F32], [2]>:$out);                 ← ...  and so is out

  let assemblyFormat =
    "$a `,` $b `,` $out attr-dict `:` type($a) `,` type($b) `,` type($out)";

  let hasVerifier = 1;
}
```

Solution — ODS

```
def Blas_MatMulOp : Blas_Op<"matmul"> {
  let summary = "matrix-matrix product: out += a * b";

  let arguments = (ins
    MemRefRankOf<[F32], [2]>:$a,
    MemRefRankOf<[F32], [2]>:$b,
    MemRefRankOf<[F32], [2]>:$out);

  let assemblyFormat =
    "$a `,` $b `,` $out attr-dict `:` type($a) `,` type($b) `,` type($out)";

  let hasVerifier = 1;
}
```

- The generated C++ class is `blas::MatMulOp`, with `getA()`, `getB()` and `getOut()` — the names the lowering expects.

Solution — Verifier

```
LogicalResult MatMulOp::verify() {
  auto aType = cast<MemRefType>(getA().getType());
  auto bType = cast<MemRefType>(getB().getType());
  auto outType = cast<MemRefType>(getOut().getType());

  if (failed(verifyDimsMatch(*this, aType.getDimSize(1), bType.getDimSize(0),
                             "inner dimensions must match")))
    return failure();
  if (failed(verifyDimsMatch(*this, aType.getDimSize(0), outType.getDimSize(0),
                             "row count of 'a' must match row count of 'out'")))
    return failure();
  return verifyDimsMatch(*this, bType.getDimSize(1), outType.getDimSize(1),
                         "column count of 'b' must match column count of 'out'");
}
```

constraint	why
`a.dim(1) == b.dim(0)`	the reduction dimension
`a.dim(0) == out.dim(0)`	rows
`b.dim(1) == out.dim(1)`	columns

What the Build Does

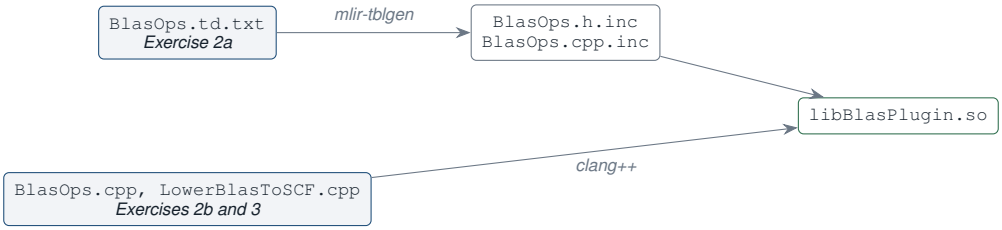
`BlasOps.td.txt`
Exercise 2a

What the Build Does



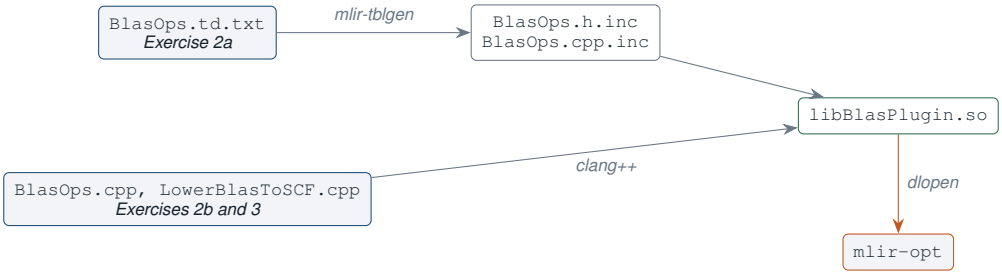
- `mlir-tblgen` turns the ODS into C++: the op's class (`BlasOps.h.inc`) and its definitions (`BlasOps.cpp.inc`).

What the Build Does



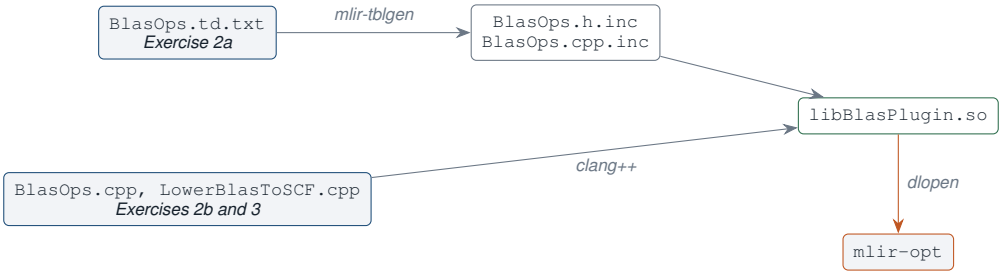
- `mlir-tblgen` turns the ODS into C++: the op's class (`BlasOps.h.inc`) and its definitions (`BlasOps.cpp.inc`).
- `clang++` compiles the generated and the hand-written C++ into the plugin.

What the Build Does



- `mlir-tblgen` turns the ODS into C++: the op's class (`BlasOps.h.inc`) and its definitions (`BlasOps.cpp.inc`).
- `clang++` compiles the generated and the hand-written C++ into the plugin.
- `mlir-opt` loads the plugin at run time.

What the Build Does



- `mlir-tblgen` turns the ODS into C++: the op's class (`BlasOps.h.inc`) and its definitions (`BlasOps.cpp.inc`).
- `clang++` compiles the generated and the hand-written C++ into the plugin.
- `mlir-opt` loads the plugin at run time.
- After editing a `.td.txt` or `.cpp` file, rebuild the plugin (cell 1.31) before round-tripping.

Check 2 — Round-Trip through the Plugin

```
!mlir-opt --load-dialect-plugin=libBlasPlugin.so matmul_custom.mlir
```

▶ Colab 1.32–1.34

Check 2 — Round-Trip through the Plugin

```
!mlir-opt --load-dialect-plugin=libBlasPlugin.so matmul_custom.mlir
```

- Success looks like `blas.matmul` printed back in the same syntax.

Check 2 — Round-Trip through the Plugin

```
!mlir-opt --load-dialect-plugin=libBlasPlugin.so matmul_custom.mlir
```

- Success looks like `blas.matmul` printed back in the same syntax.

```
matmul_custom.mlir:5:3: error: custom op 'blas.matmul' is unknown
```

Either the op is not defined — or the plugin did not rebuild.

Step 3 — Lowering `blas` to Loops

From `blas.matvec` to Loops

```
blas.matvec %mat, %vec, %out
```

convert-blas-to-scf

```
scf.for %i = %c0 to %numRows step %c1 {
  %init = memref.load %out[%i]
  %acc = scf.for %j = ...  iter_args(%a = %init) {
    ...
  }
  memref.store %acc, %out[%i]
}
```

From `blas.matvec` to Loops

```
blas.matvec %mat, %vec, %out
```

convert-blas-to-scf

```
scf.for %i = %c0 to %numRows step %c1 {
  %init = memref.load %out[%i]
  %acc = scf.for %j = ...  iter_args(%a = %init) {
    ...
  }
  memref.store %acc, %out[%i]
}
```

- The pass `convert-blas-to-scf`, in `LowerBlasToSCF.cpp`, lowers each `blas` op to `scf.for` loops.

From `blas.matvec` to Loops

```
blas.matvec %mat, %vec, %out
```

convert-blas-to-scf

```
scf.for %i = %c0 to %numRows step %c1 {
  %init = memref.load %out[%i]
  %acc = scf.for %j = ...  iter_args(%a = %init) {
    ...
  }
  memref.store %acc, %out[%i]
}
```

- The pass `convert-blas-to-scf`, in `LowerBlasToSCF.cpp`, lowers each `blas` op to `scf.for` loops.
- It is a set of rewrite patterns, one per op: `MatVecToSCFPattern` for `blas.matvec`.

From `blas.matvec` to Loops

```
blas.matvec %mat, %vec, %out
```

convert-blas-to-scf

```
scf.for %i = %c0 to %numRows step %c1 {
  %init = memref.load %out[%i]
  %acc = scf.for %j = ...  iter_args(%a = %init) {
    ...
  }
  memref.store %acc, %out[%i]
}
```

- The pass `convert-blas-to-scf`, in `LowerBlasToSCF.cpp`, lowers each `blas` op to `scf.for` loops.
- It is a set of rewrite patterns, one per op: `MatVecToSCFPattern` for `blas.matvec`.
- Exercise 3 adds `MatMulToSCFPattern`, which builds the loops from Step 1.

▶ Colab 1.36

Reference: `MatVecToSCFPattern`, Step 1 of 6 — Scaffolding

the C++

```
struct MatVecToSCFPattern : public OpRewritePattern<MatVecOp> {
  using OpRewritePattern::OpRewritePattern;
  LogicalResult
  matchAndRewrite(MatVecOp op,
                  PatternRewriter &rewriter) const override {
    Location loc = op.getLoc();
    Value mat = op.getMat();

    Value c0 = arith::ConstantIndexOp::create(rewriter, loc, 0);
    Value c1 = arith::ConstantIndexOp::create(rewriter, loc, 1);
    Value numRows = memref::DimOp::create(rewriter, loc, mat, c0);
    Value numCols = memref::DimOp::create(rewriter, loc, mat, c1);
    // steps 2-4 go here
    rewriter.eraseOp(op);
    return success();
  }
};
```

the IR it generates

Reference: `MatVecToSCFPattern`, Step 1 of 6 — Scaffolding

the C++

```
struct MatVecToSCFPattern : public OpRewritePattern<MatVecOp> {
  using OpRewritePattern::OpRewritePattern;
  LogicalResult
  matchAndRewrite(MatVecOp op,
                  PatternRewriter &rewriter) const override {
    Location loc = op.getLoc();
    Value mat = op.getMat();

    Value c0 = arith::ConstantIndexOp::create(rewriter, loc, 0);
    Value c1 = arith::ConstantIndexOp::create(rewriter, loc, 1);
    Value numRows = memref::DimOp::create(rewriter, loc, mat, c0);
    Value numCols = memref::DimOp::create(rewriter, loc, mat, c1);
    // steps 2-4 go here
    rewriter.eraseOp(op);
    return success();
  }
};
```

the IR it generates

- A pattern matches one op: `OpRewritePattern<MatVecOp>` rewrites `blas.matvec` and nothing else.

Reference: `MatVecToSCFPattern`, Step 1 of 6 — Scaffolding

the C++

```
struct MatVecToSCFPattern : public OpRewritePattern<MatVecOp> {
  using OpRewritePattern::OpRewritePattern;
  LogicalResult
  matchAndRewrite(MatVecOp op,
                  PatternRewriter &rewriter) const override {
    Location loc = op.getLoc();
    Value mat = op.getMat();

    Value c0 = arith::ConstantIndexOp::create(rewriter, loc, 0);
    Value c1 = arith::ConstantIndexOp::create(rewriter, loc, 1);
    Value numRows = memref::DimOp::create(rewriter, loc, mat, c0);
    Value numCols = memref::DimOp::create(rewriter, loc, mat, c1);
    // steps 2-4 go here
    rewriter.eraseOp(op);
    return success();
  }
};
```

the IR it generates

```
%c0 = arith.constant 0 : index
%c1 = arith.constant 1 : index
%numRows = memref.dim %mat, %c0
%numCols = memref.dim %mat, %c1
```

- A pattern matches one op: `OpRewritePattern<MatVecOp>` rewrites `blas.matvec` and nothing else.
- Then the dimensions of the matrix, with `memref.dim`: `c0` and `c1` say which dimension.

Reference: `MatVecToSCFPattern`, Step 2 of 6 — Row Loop

the C++

```
auto rowLoop = scf::ForOp::create(rewriter, loc, c0, numRows, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(rowLoop.getBody());
  Value i = rowLoop.getInductionVar();

  // steps 3 and 4 go here
}
```

the IR it generates

```
scf.for %i = %c0 to %numRows step %c1 {


  // steps 3 and 4 go here
}
```

Reference: `MatVecToSCFPattern`, Step 2 of 6 — Row Loop

the C++

```
auto rowLoop = scf::ForOp::create(rewriter, loc, c0, numRows, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(rowLoop.getBody());
  Value i = rowLoop.getInductionVar();

  // steps 3 and 4 go here
}
```

the IR it generates

```
scf.for %i = %c0 to %numRows step %c1 {


  // steps 3 and 4 go here
}
```

- Each `ForOp::create` is one `scf.for`.

Reference: `MatVecToSCFPattern`, Step 2 of 6 — Row Loop

the C++

```
auto rowLoop = scf::ForOp::create(rewriter, loc, c0, numRows, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(rowLoop.getBody());
  Value i = rowLoop.getInductionVar();

  // steps 3 and 4 go here
}
```

the IR it generates

```
scf.for %i = %c0 to %numRows step %c1 {


  // steps 3 and 4 go here
}
```

- Each `ForOp::create` is one `scf.for`.
- Ops created after `setInsertionPointToStart` land inside that loop's body.

Reference: `MatVecToSCFPattern`, Step 2 of 6 — Row Loop

the C++

```
auto rowLoop = scf::ForOp::create(rewriter, loc, c0, numRows, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(rowLoop.getBody());
  Value i = rowLoop.getInductionVar();

  // steps 3 and 4 go here
}
```

the IR it generates

```
scf.for %i = %c0 to %numRows step %c1 {


  // steps 3 and 4 go here
}
```

- Each `ForOp::create` is one `scf.for`.
- Ops created after `setInsertionPointToStart` land inside that loop's body.
- `InsertionGuard` restores the insertion point when the scope ends.

Reference: `MatVecToSCFPattern`, Step 3 of 6 — Reduction Loop

the C++

```
Value init = memref::LoadOp::create(rewriter, loc, out,
                                    ValueRange{i});
auto redLoop = scf::ForOp::create(rewriter, loc, c0, numCols, c1,
                                  ValueRange{init});
{
  OpBuilder::InsertionGuard innerGuard(rewriter);
  rewriter.setInsertionPointToStart(redLoop.getBody());
  Value j = redLoop.getInductionVar();
  Value acc = redLoop.getRegionIterArgs()[0];

  Value matElem = memref::LoadOp::create(rewriter, loc, mat,
                                         ValueRange{i, j});
  Value vecElem = memref::LoadOp::create(rewriter, loc, vec,
                                         ValueRange{j});
  Value prod =
      arith::MulFOp::create(rewriter, loc, matElem, vecElem);
  Value sum = arith::AddFOp::create(rewriter, loc, acc, prod);
  scf::YieldOp::create(rewriter, loc, ValueRange{sum});
}
```

the IR it generates

Reference: `MatVecToSCFPattern`, Step 3 of 6 — Reduction Loop

the C++

```
Value init = memref::LoadOp::create(rewriter, loc, out,
                                    ValueRange{i});
auto redLoop = scf::ForOp::create(rewriter, loc, c0, numCols, c1,
                                  ValueRange{init});
{
  OpBuilder::InsertionGuard innerGuard(rewriter);
  rewriter.setInsertionPointToStart(redLoop.getBody());
  Value j = redLoop.getInductionVar();
  Value acc = redLoop.getRegionIterArgs()[0];

  Value matElem = memref::LoadOp::create(rewriter, loc, mat,
                                         ValueRange{i, j});
  Value vecElem = memref::LoadOp::create(rewriter, loc, vec,
                                         ValueRange{j});
  Value prod =
      arith::MulFOp::create(rewriter, loc, matElem, vecElem);
  Value sum = arith::AddFOp::create(rewriter, loc, acc, prod);
  scf::YieldOp::create(rewriter, loc, ValueRange{sum});
}
```

the IR it generates

```
%init = memref.load %out[%i]
```

Reference: `MatVecToSCFPattern`, Step 3 of 6 — Reduction Loop

the C++

```
Value init = memref::LoadOp::create(rewriter, loc, out,
                                    ValueRange{i});
auto redLoop = scf::ForOp::create(rewriter, loc, c0, numCols, c1,
                                  ValueRange{init});
{
  OpBuilder::InsertionGuard innerGuard(rewriter);
  rewriter.setInsertionPointToStart(redLoop.getBody());
  Value j = redLoop.getInductionVar();
  Value acc = redLoop.getRegionIterArgs()[0];

  Value matElem = memref::LoadOp::create(rewriter, loc, mat,
                                         ValueRange{i, j});
  Value vecElem = memref::LoadOp::create(rewriter, loc, vec,
                                         ValueRange{j});
  Value prod =
      arith::MulFOp::create(rewriter, loc, matElem, vecElem);
  Value sum = arith::AddFOp::create(rewriter, loc, acc, prod);
  scf::YieldOp::create(rewriter, loc, ValueRange{sum});
}
```

the IR it generates

```
%init = memref.load %out[%i]

%acc = scf.for %j = %c0 to %numCols step %c1
    iter_args(%a = %init) -> (f32) {




}
```

Reference: `MatVecToSCFPattern`, Step 3 of 6 — Reduction Loop

the C++

```
Value init = memref::LoadOp::create(rewriter, loc, out,
                                    ValueRange{i});
auto redLoop = scf::ForOp::create(rewriter, loc, c0, numCols, c1,
                                  ValueRange{init});
{
  OpBuilder::InsertionGuard innerGuard(rewriter);
  rewriter.setInsertionPointToStart(redLoop.getBody());
  Value j = redLoop.getInductionVar();
  Value acc = redLoop.getRegionIterArgs()[0];

  Value matElem = memref::LoadOp::create(rewriter, loc, mat,
                                         ValueRange{i, j});
  Value vecElem = memref::LoadOp::create(rewriter, loc, vec,
                                         ValueRange{j});
  Value prod =
      arith::MulFOp::create(rewriter, loc, matElem, vecElem);
  Value sum = arith::AddFOp::create(rewriter, loc, acc, prod);
  scf::YieldOp::create(rewriter, loc, ValueRange{sum});
}
```

the IR it generates

```
%init = memref.load %out[%i]

%acc = scf.for %j = %c0 to %numCols step %c1
    iter_args(%a = %init) -> (f32) {




}
```

Reference: `MatVecToSCFPattern`, Step 3 of 6 — Reduction Loop

the C++

```
Value init = memref::LoadOp::create(rewriter, loc, out,
                                    ValueRange{i});
auto redLoop = scf::ForOp::create(rewriter, loc, c0, numCols, c1,
                                  ValueRange{init});
{
  OpBuilder::InsertionGuard innerGuard(rewriter);
  rewriter.setInsertionPointToStart(redLoop.getBody());
  Value j = redLoop.getInductionVar();
  Value acc = redLoop.getRegionIterArgs()[0];

  Value matElem = memref::LoadOp::create(rewriter, loc, mat,
                                         ValueRange{i, j});
  Value vecElem = memref::LoadOp::create(rewriter, loc, vec,
                                         ValueRange{j});
  Value prod =
      arith::MulFOp::create(rewriter, loc, matElem, vecElem);
  Value sum = arith::AddFOp::create(rewriter, loc, acc, prod);
  scf::YieldOp::create(rewriter, loc, ValueRange{sum});
}
```

the IR it generates

```
%init = memref.load %out[%i]

%acc = scf.for %j = %c0 to %numCols step %c1
    iter_args(%a = %init) -> (f32) {

  %m = memref.load %mat[%i, %j]

  %v = memref.load %vec[%j]

}
```

Reference: `MatVecToSCFPattern`, Step 3 of 6 — Reduction Loop

the C++

```
Value init = memref::LoadOp::create(rewriter, loc, out,
                                    ValueRange{i});
auto redLoop = scf::ForOp::create(rewriter, loc, c0, numCols, c1,
                                  ValueRange{init});
{
  OpBuilder::InsertionGuard innerGuard(rewriter);
  rewriter.setInsertionPointToStart(redLoop.getBody());
  Value j = redLoop.getInductionVar();
  Value acc = redLoop.getRegionIterArgs()[0];

  Value matElem = memref::LoadOp::create(rewriter, loc, mat,
                                         ValueRange{i, j});
  Value vecElem = memref::LoadOp::create(rewriter, loc, vec,
                                         ValueRange{j});
  Value prod =
      arith::MulFOp::create(rewriter, loc, matElem, vecElem);
  Value sum = arith::AddFOp::create(rewriter, loc, acc, prod);
  scf::YieldOp::create(rewriter, loc, ValueRange{sum});
}
```

the IR it generates

```
%init = memref.load %out[%i]

%acc = scf.for %j = %c0 to %numCols step %c1
    iter_args(%a = %init) -> (f32) {

  %m = memref.load %mat[%i, %j]

  %v = memref.load %vec[%j]

  %prod = arith.mulf %m, %v : f32

  %a2 = arith.addf %a, %prod : f32

}
```

Reference: `MatVecToSCFPattern`, Step 3 of 6 — Reduction Loop

the C++

```
Value init = memref::LoadOp::create(rewriter, loc, out,
                                    ValueRange{i});
auto redLoop = scf::ForOp::create(rewriter, loc, c0, numCols, c1,
                                  ValueRange{init});
{
  OpBuilder::InsertionGuard innerGuard(rewriter);
  rewriter.setInsertionPointToStart(redLoop.getBody());
  Value j = redLoop.getInductionVar();
  Value acc = redLoop.getRegionIterArgs()[0];

  Value matElem = memref::LoadOp::create(rewriter, loc, mat,
                                         ValueRange{i, j});
  Value vecElem = memref::LoadOp::create(rewriter, loc, vec,
                                         ValueRange{j});
  Value prod =
      arith::MulFOp::create(rewriter, loc, matElem, vecElem);
  Value sum = arith::AddFOp::create(rewriter, loc, acc, prod);
  scf::YieldOp::create(rewriter, loc, ValueRange{sum});
}
```

the IR it generates

```
%init = memref.load %out[%i]

%acc = scf.for %j = %c0 to %numCols step %c1
    iter_args(%a = %init) -> (f32) {

  %m = memref.load %mat[%i, %j]

  %v = memref.load %vec[%j]

  %prod = arith.mulf %m, %v : f32

  %a2 = arith.addf %a, %prod : f32
  scf.yield %a2 : f32
}
```

Reference: `MatVecToSCFPattern`, Step 4 of 6 — Store

the C++

```
LogicalResult matchAndRewrite(MatVecOp op, ...) const override {
  // step 1: mat, numRows, numCols, c0, c1
  auto rowLoop = scf::ForOp::create(rewriter, loc, c0, numRows, c1);
  {
    // step 2: the guard, the insertion point, i
    Value init = memref::LoadOp::create(..., out, ValueRange{i});
    auto redLoop = scf::ForOp::create(..., ValueRange{init});
    {
      // step 3: the reduction body
    }
    memref::StoreOp::create(rewriter, loc, redLoop.getResult(0), out,
                            ValueRange{i});
  }
  rewriter.eraseOp(op);
  return success();
}
```

the IR it generates

```
scf.for %i = %c0 to %numRows step %c1 {

  %init = memref.load %out[%i]
  %acc = scf.for %j ... {

    ...
  }
  memref.store %acc, %out[%i]

}
```

- The store takes the result of the `scf.for`, `getResult(0)`: the value of the reduction, `%acc`.

Reference: `MatVecToSCFPattern`, Step 5 of 6 — Pass Declaration

`blas-dialect/include/Blas/Passes.td`:

```
def LowerBlasToSCFPass : Pass<"convert-blas-to-scf", "::mlir::func::FuncOp"> {
  let summary = "Lower blas ops to scf.for loops";
  let dependentDialects = [
    "::mlir::scf::SCFDialect",
    "::mlir::memref::MemRefDialect",
    "::mlir::arith::ArithDialect",
  ];
}
```

Reference: `MatVecToSCFPattern`, Step 5 of 6 — Pass Declaration

`blas-dialect/include/Blas/Passes.td`:

```
def LowerBlasToSCFPass : Pass<"convert-blas-to-scf", "::mlir::func::FuncOp"> {
  let summary = "Lower blas ops to scf.for loops";
  let dependentDialects = [
    "::mlir::scf::SCFDialect",
    "::mlir::memref::MemRefDialect",
    "::mlir::arith::ArithDialect",
  ];
}
```

- `"convert-blas-to-scf"` is the pass's name on the `mlir-opt` command line.

Reference: `MatVecToSCFPattern`, Step 5 of 6 — Pass Declaration

`blas-dialect/include/Blas/Passes.td`:

```
def LowerBlasToSCFPass : Pass<"convert-blas-to-scf", "::mlir::func::FuncOp"> {
  let summary = "Lower blas ops to scf.for loops";
  let dependentDialects = [
    "::mlir::scf::SCFDialect",
    "::mlir::memref::MemRefDialect",
    "::mlir::arith::ArithDialect",
  ];
}
```

- `"convert-blas-to-scf"` is the pass's name on the `mlir-opt` command line.
- `::mlir::func::FuncOp` makes it a function-level pass: `runOnOperation()` runs on each `func.func`.

Reference: `MatVecToSCFPattern`, Step 5 of 6 — Pass Declaration

`blas-dialect/include/Blas/Passes.td`:

```
def LowerBlasToSCFPass : Pass<"convert-blas-to-scf", "::mlir::func::FuncOp"> {
  let summary = "Lower blas ops to scf.for loops";
  let dependentDialects = [
    "::mlir::scf::SCFDialect",
    "::mlir::memref::MemRefDialect",
    "::mlir::arith::ArithDialect",
  ];
}
```

- `"convert-blas-to-scf"` is the pass's name on the `mlir-opt` command line.
- `::mlir::func::FuncOp` makes it a function-level pass: `runOnOperation()` runs on each `func.func`.
- `dependentDialects` lists the dialects whose ops the pass creates.

Reference: `MatVecToSCFPattern`, Step 5 of 6 — Pass Declaration

`blas-dialect/include/Blas/Passes.td`:

```
def LowerBlasToSCFPass : Pass<"convert-blas-to-scf", "::mlir::func::FuncOp"> {
  let summary = "Lower blas ops to scf.for loops";
  let dependentDialects = [
    "::mlir::scf::SCFDialect",
    "::mlir::memref::MemRefDialect",
    "::mlir::arith::ArithDialect",
  ];
}
```

- `"convert-blas-to-scf"` is the pass's name on the `mlir-opt` command line.
- `::mlir::func::FuncOp` makes it a function-level pass: `runOnOperation()` runs on each `func.func`.
- `dependentDialects` lists the dialects whose ops the pass creates.
- `mlir-tblgen` generates a base class, `LowerBlasToSCFPassBase`, and the pass derives from it.

Reference: `MatVecToSCFPattern`, Step 6 of 6 — Pattern Registration

`LowerBlasToSCF.cpp`, the pass the pattern belongs to:

```
struct LowerBlasToSCFPass
    : public blas::impl::LowerBlasToSCFPassBase<LowerBlasToSCFPass> {
  void runOnOperation() override {
    RewritePatternSet patterns(&getContext());
    patterns.add<MatVecToSCFPattern>(&getContext());
    // TODO(exercise): register the matmul pattern too:
    //   patterns.add<MatMulToSCFPattern>(&getContext());

    if (failed(applyPatternsGreedily(getOperation(), std::move(patterns)))) {
      signalPassFailure();
      return;
    }
    ...
  }
};
```

Reference: `MatVecToSCFPattern`, Step 6 of 6 — Pattern Registration

`LowerBlasToSCF.cpp`, the pass the pattern belongs to:

```
struct LowerBlasToSCFPass
    : public blas::impl::LowerBlasToSCFPassBase<LowerBlasToSCFPass> {
  void runOnOperation() override {
    RewritePatternSet patterns(&getContext());
    patterns.add<MatVecToSCFPattern>(&getContext());
    // TODO(exercise): register the matmul pattern too:
    //   patterns.add<MatMulToSCFPattern>(&getContext());

    if (failed(applyPatternsGreedily(getOperation(), std::move(patterns)))) {
      signalPassFailure();
      return;
    }
    ...
  }
};
```

- The base class generated from `Passes.td`; the plugin registers the pass with `mlir-opt`.

Reference: `MatVecToSCFPattern`, Step 6 of 6 — Pattern Registration

`LowerBlasToSCF.cpp`, the pass the pattern belongs to:

```
struct LowerBlasToSCFPass
    : public blas::impl::LowerBlasToSCFPassBase<LowerBlasToSCFPass> {
  void runOnOperation() override {
    RewritePatternSet patterns(&getContext());
    patterns.add<MatVecToSCFPattern>(&getContext());
    // TODO(exercise): register the matmul pattern too:
    //   patterns.add<MatMulToSCFPattern>(&getContext());

    if (failed(applyPatternsGreedily(getOperation(), std::move(patterns)))) {
      signalPassFailure();
      return;
    }
    ...
  }
};
```

- The base class generated from `Passes.td`; the plugin registers the pass with `mlir-opt`.
- `patterns.add<...>` registers a pattern with the pass. `MatMulToSCFPattern` goes on the same line.

Reference: `MatVecToSCFPattern`, Step 6 of 6 — Pattern Registration

`LowerBlasToSCF.cpp`, the pass the pattern belongs to:

```
struct LowerBlasToSCFPass
    : public blas::impl::LowerBlasToSCFPassBase<LowerBlasToSCFPass> {
  void runOnOperation() override {
    RewritePatternSet patterns(&getContext());
    patterns.add<MatVecToSCFPattern>(&getContext());
    // TODO(exercise): register the matmul pattern too:
    //   patterns.add<MatMulToSCFPattern>(&getContext());

    if (failed(applyPatternsGreedily(getOperation(), std::move(patterns)))) {
      signalPassFailure();
      return;
    }
    ...
  }
};
```

- The base class generated from `Passes.td`; the plugin registers the pass with `mlir-opt`.
- `patterns.add<...>` registers a pattern with the pass. `MatMulToSCFPattern` goes on the same line.
- `applyPatternsGreedily` applies them to the IR the pass runs on — next slide.

How Patterns Are Applied

- **The driver applies the patterns.** `applyPatternsGreedily` keeps applying the registered patterns until none of them matches any more.

How Patterns Are Applied

- **The driver applies the patterns.** `applyPatternsGreedily` keeps applying the registered patterns until none of them matches any more.
- **Every IR change goes through the `PatternRewriter`.** Creating, replacing and erasing ops all go through it, so the driver sees each change and can revisit what it affected. A pattern never edits the IR behind its back.

How Patterns Are Applied

- **The driver applies the patterns.** `applyPatternsGreedily` keeps applying the registered patterns until none of them matches any more.
- **Every IR change goes through the `PatternRewriter`.** Creating, replacing and erasing ops all go through it, so the driver sees each change and can revisit what it affected. A pattern never edits the IR behind its back.
- **`matchAndRewrite` reports what it did.** It returns `success()` when it rewrote the op and `failure()` when the op does not match — and on `failure()` it must leave the IR untouched.

Exercise 3 — Writing the Lowering

`MatMulToSCFPattern` in
`blas-dialect/lib/LowerBlasToSCF.cpp`

- One pass, `convert-blas-to-scf`, one pattern per op.

Exercise 3 — Writing the Lowering

`MatMulToSCFPattern` in
`blas-dialect/lib/LowerBlasToSCF.cpp`

- One pass, `convert-blas-to-scf`, one pattern per op.
- The scaffolding is already there, commented out: operands, index constants, problem sizes, and the `eraseOp` that retires the op.

Exercise 3 — Writing the Lowering

`MatMulToSCFPattern` in
`blas-dialect/lib/LowerBlasToSCF.cpp`

- One pass, `convert-blas-to-scf`, one pattern per op.
- The scaffolding is already there, commented out: operands, index constants, problem sizes, and the `eraseOp` that retires the op.
- Write the loop nest — the same one as step 1, now built programmatically — and then register the pattern.

Exercise 3 — Writing the Lowering

`MatMulToSCFPattern` in
`blas-dialect/lib/LowerBlasToSCF.cpp`

- One pass, `convert-blas-to-scf`, one pattern per op.
- The scaffolding is already there, commented out: operands, index constants, problem sizes, and the `eraseOp` that retires the op.
- Write the loop nest — the same one as step 1, now built programmatically — and then register the pattern.
- Reference: `MatVecToSCFPattern`, same file, complete.

Exercise 3 — Writing the Lowering

`MatMulToSCFPattern` in
`blas-dialect/lib/LowerBlasToSCF.cpp`

- One pass, `convert-blas-to-scf`, one pattern per op.
- The scaffolding is already there, commented out: operands, index constants, problem sizes, and the `eraseOp` that retires the op.
- Write the loop nest — the same one as step 1, now built programmatically — and then register the pattern.
- Reference: `MatVecToSCFPattern`, same file, complete.

10 minutes to write it — then we walk through the solution.

Solution — `MatMulToSCFPattern`

```
auto rowLoop = scf::ForOp::create(rewriter, loc, c0, M, c1);
{
  OpBuilder::InsertionGuard rowGuard(rewriter);
  rewriter.setInsertionPointToStart(rowLoop.getBody());
  Value i = rowLoop.getInductionVar();

  auto colLoop = scf::ForOp::create(rewriter, loc, c0, N, c1);
  {
    OpBuilder::InsertionGuard colGuard(rewriter);
    rewriter.setInsertionPointToStart(colLoop.getBody());
    Value j = colLoop.getInductionVar();

    Value init = memref::LoadOp::create(rewriter, loc, out, ValueRange{i, j});
    auto redLoop = scf::ForOp::create(rewriter, loc, c0, K, c1, ValueRange{init});
    {
      // the guard, the insertion point, k and acc: as in matvec
      Value aElem = memref::LoadOp::create(rewriter, loc, a, ValueRange{i, k});
      Value bElem = memref::LoadOp::create(rewriter, loc, b, ValueRange{k, j});
      // prod, sum and the yield: as in matvec
    }
    memref::StoreOp::create(rewriter, loc, redLoop.getResult(0), out,
                            ValueRange{i, j});
  }
}
```

Solution — `MatMulToSCFPattern`

```
auto rowLoop = scf::ForOp::create(rewriter, loc, c0, M, c1);
{
  OpBuilder::InsertionGuard rowGuard(rewriter);
  rewriter.setInsertionPointToStart(rowLoop.getBody());
  Value i = rowLoop.getInductionVar();

  auto colLoop = scf::ForOp::create(rewriter, loc, c0, N, c1);  ← one more loop:  columns
  {
    OpBuilder::InsertionGuard colGuard(rewriter);
    rewriter.setInsertionPointToStart(colLoop.getBody());
    Value j = colLoop.getInductionVar();

    Value init = memref::LoadOp::create(rewriter, loc, out, ValueRange{i, j});
    auto redLoop = scf::ForOp::create(rewriter, loc, c0, K, c1, ValueRange{init});
    {
      // the guard, the insertion point, k and acc: as in matvec
      Value aElem = memref::LoadOp::create(rewriter, loc, a, ValueRange{i, k});
      Value bElem = memref::LoadOp::create(rewriter, loc, b, ValueRange{k, j});
      // prod, sum and the yield: as in matvec
    }
    memref::StoreOp::create(rewriter, loc, redLoop.getResult(0), out,
                            ValueRange{i, j});
  }
}
```

Solution — `MatMulToSCFPattern`

```
auto rowLoop = scf::ForOp::create(rewriter, loc, c0, M, c1);
{
  OpBuilder::InsertionGuard rowGuard(rewriter);
  rewriter.setInsertionPointToStart(rowLoop.getBody());
  Value i = rowLoop.getInductionVar();

  auto colLoop = scf::ForOp::create(rewriter, loc, c0, N, c1);
  {
    OpBuilder::InsertionGuard colGuard(rewriter);
    rewriter.setInsertionPointToStart(colLoop.getBody());
    Value j = colLoop.getInductionVar();

    Value init = memref::LoadOp::create(rewriter, loc, out, ValueRange{i, j});
    auto redLoop = scf::ForOp::create(rewriter, loc, c0, K, c1, ValueRange{init});
    {
      // the guard, the insertion point, k and acc: as in matvec
      Value aElem = memref::LoadOp::create(rewriter, loc, a, ValueRange{i, k});
      Value bElem = memref::LoadOp::create(rewriter, loc, b, ValueRange{k, j});
      // prod, sum and the yield: as in matvec
    }
    memref::StoreOp::create(rewriter, loc, redLoop.getResult(0), out,
                            ValueRange{i, j});              ← out and b take two indices
  }
}
```

Solution — Register the Pattern

```
void runOnOperation() override {
  RewritePatternSet patterns(&getContext());
  patterns.add<MatVecToSCFPattern, MatMulToSCFPattern>(&getContext());
  ...
}
```

Solution — Register the Pattern

```
void runOnOperation() override {
  RewritePatternSet patterns(&getContext());
  patterns.add<MatVecToSCFPattern, MatMulToSCFPattern>(&getContext());
  ...
}
```

- A pattern that is never registered is never applied, and `blas.matmul` survives the pass untouched.

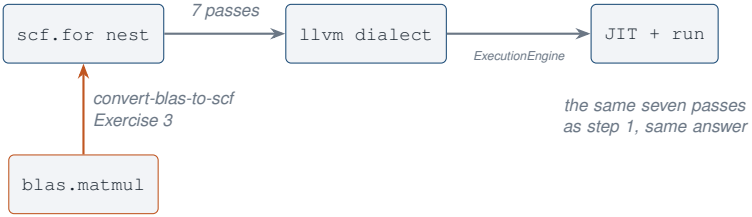
Solution — Register the Pattern

```
void runOnOperation() override {
  RewritePatternSet patterns(&getContext());
  patterns.add<MatVecToSCFPattern, MatMulToSCFPattern>(&getContext());
  ...
}
```

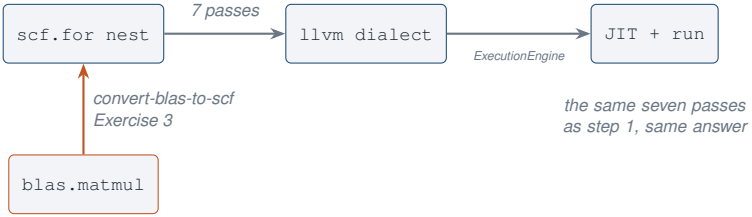
- A pattern that is never registered is never applied, and `blas.matmul` survives the pass untouched.
- Which is the same symptom as a pattern that returns `failure()`. The pass catches both:

```
error: convert-blas-to-scf did not convert 'blas.matmul' -- is its pattern
written and registered in blas-dialect/lib/LowerBlasToSCF.cpp?
```

Recap

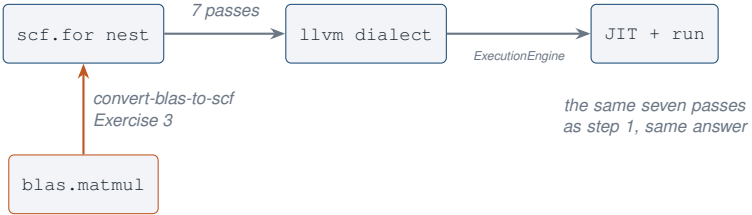

scf.for nest
7 passes
llvm dialect
ExecutionEngine
JIT + run
convert-blas-to-scf
Exercise 3
blas.matmul
the same seven passes
as step 1, same answer

Recap



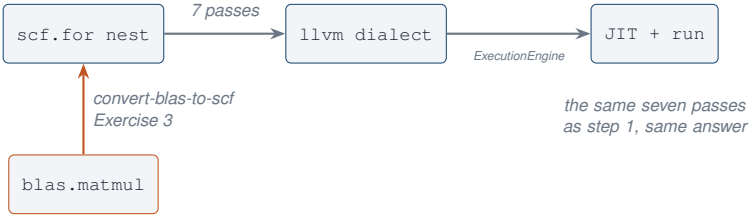
- `convert-blas-to-scf` turns `blas.matmul` into the `scf.for` nest written by hand in step 1.

Recap



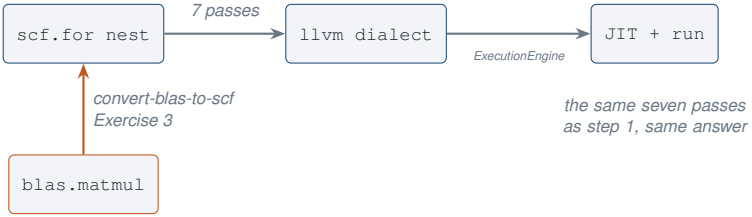
- `convert-blas-to-scf` turns `blas.matmul` into the `scf.for` nest written by hand in step 1.
- An op — operands, types, assembly syntax — from eight lines of ODS.

Recap



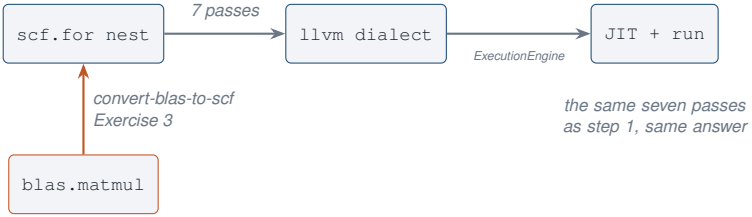
- `convert-blas-to-scf` turns `blas.matmul` into the `scf.for` nest written by hand in step 1.
- An op — operands, types, assembly syntax — from eight lines of ODS.
- A verifier, so bad IR is rejected with the op's own error message.

Recap



- `convert-blas-to-scf` turns `blas.matmul` into the `scf.for` nest written by hand in step 1.
- An op — operands, types, assembly syntax — from eight lines of ODS.
- A verifier, so bad IR is rejected with the op's own error message.
- A pass that rewrites the op into standard dialects.

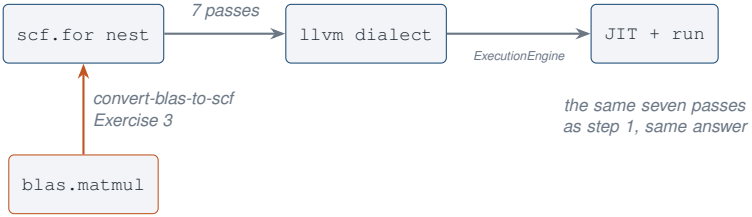
Recap



- `convert-blas-to-scf` turns `blas.matmul` into the `scf.for` nest written by hand in step 1.
- An op — operands, types, assembly syntax — from eight lines of ODS.
- A verifier, so bad IR is rejected with the op's own error message.
- A pass that rewrites the op into standard dialects.

That is a complete dialect: definition, validation, meaning.

Recap



- `convert-blas-to-scf` turns `blas.matmul` into the `scf.for` nest written by hand in step 1.
- An op — operands, types, assembly syntax — from eight lines of ODS.
- A verifier, so bad IR is rejected with the op's own error message.
- A pass that rewrites the op into standard dialects.

That is a complete dialect: definition, validation, meaning.

Everything upstream MLIR does — `linalg`, `vector`, `gpu` — is this, at scale.